

SCHOOL MANAGEMENT SYSTEM

A Project Report

Submitted in partial fulfilment of the requirements for the award of the
degree

Programme: _____

Submitted By

Registration Number: _____

Centre / Department: _____

Institution / University: _____

Campus / Location: _____

Academic Year: _____

Declaration by Student

I, _____, Registration Number _____, hereby declare that this project report titled School Management System is based on the work carried out by me as part of my academic project. The report explains the purpose, design, implementation, testing and future scope of the system in my own words.

The information included in this report has been prepared from my project implementation, learning notes and technical documentation. Wherever external tools, libraries or official documentation were referred to, they have been acknowledged in the references section.

Name of Student: _____

Registration Number: _____

Programme: _____

Signature: _____

Date: _____

Certificate

This is to certify that the project report titled School Management System has been prepared and submitted by _____, Registration Number _____, in partial fulfilment of the academic requirements of the programme.

The project presents a web-based system for managing school administration, academic records, attendance, assignments, fees, results and notifications. The work demonstrates practical use of the MERN stack and related development tools.

Student Name: _____

Registration Number: _____

Programme: _____

Project Guide / Faculty Signature: _____

Guide / Faculty Name: _____

Date: _____

Acknowledgement

I am thankful to my institution and department for giving me the opportunity to prepare this project report and apply my software development knowledge to a practical problem.

I would also like to thank my faculty members and project guide for their guidance, feedback and encouragement during the work. Their suggestions helped me think more clearly about project structure, documentation, security and testing.

I am grateful to my family and friends for their support throughout the development process. Finally, I acknowledge the open-source tools and official documentation used during the project, especially the resources related to React, Node.js, Express, MongoDB, Redux Toolkit, Tailwind CSS and other supporting libraries.

Abstract

The School Management System is a web-based application developed to simplify common administrative and academic tasks in a school. In many institutions, student records, attendance, fee details, assignments and examination results are still handled through paper files, spreadsheets or disconnected software. This often leads to duplicate work, delayed updates and difficulty in tracking information accurately.

The system proposed in this report brings these activities into a single digital platform. It provides role-based access for administrators, teachers and students so that each user can work only with the features relevant to them. The application includes modules for user management, student and teacher profiles, attendance, assignments, submissions, examinations, results, fees, expenses, notifications and document generation.

The frontend is built with React.js and Vite, using Tailwind CSS, Shadcn UI and Redux Toolkit for a responsive and manageable user interface. The backend uses Node.js and Express.js, while MongoDB with Mongoose is used for data storage. Security is handled using JSON Web Tokens and Argon2 password hashing. The project also uses supporting tools such as Cloudinary for media storage, Twilio for SMS notifications, PDFKit for PDF generation and QRCode for verification-related documents.

Overall, this project shows how a school can reduce manual work, improve record accuracy and provide faster access to important academic and financial information. The system is suitable as a working prototype and can be expanded further with online payments, mobile/PWA features and analytics.

Table of Contents

Declaration by Student	2
Certificate	3
Acknowledgement	4
Abstract	5
List of Tables	9
List of Figures	9
List of Abbreviations	10
Chapter 1: Introduction	11
1.1 Project Overview	11
1.2 Background of the Problem	11
1.3 Problem Statement	11
1.4 Aim of the Project	11
1.5 Objectives of the Project	11
1.6 Scope of the Project	12
1.7 Importance and Applicability	12
1.8 Project Type and Category	12
1.9 Organization of the Report	12
Chapter 2: Review of Literature and Existing Systems	13
2.1 Introduction	13
2.2 Existing Methods	13
2.3 Related Technologies	13
2.4 Comparative Study	13
2.5 Research Gap / System Gap	14

2.6 Need of the Proposed System	14
2.7 Summary	14
Chapter 3: Requirement Analysis	15
3.1 Introduction	15
3.2 Stakeholders	15
3.3 Requirement Summary	15
3.4 Functional Requirements	15
3.5 Non-Functional Requirements	16
3.6 Hardware and Software Requirements	16
3.7 Use Case Overview	16
Chapter 4: System Design	17
4.1 Introduction	17
4.2 Proposed System Architecture	17
4.3 Technology Stack	18
4.4 Database Design	18
4.5 Role-Based Access Control	18
4.6 Data Flow	19
Chapter 5: Implementation	20
5.1 Introduction	20
5.2 Backend Structure	20
5.3 Frontend Structure	20
5.4 Authentication Workflow	20
5.5 Main Modules Implemented	21
5.6 Security Measures	21
5.7 Development Methodology	22

Chapter 6: Testing, Results and Discussion	23
6.1 Introduction	23
6.2 Output Screens to Attach	23
6.3 Test Cases Summary	23
6.4 Functional Results	24
6.5 User Interface Results	24
6.6 Performance Observations	24
6.7 Discussion	24
6.8 Limitations	24
Chapter 7: Conclusion and Future Scope	25
7.1 Conclusion	25
7.2 Achievement of Objectives	25
7.3 Major Learnings	25
7.4 Practical Applicability	25
7.5 Future Scope	26
7.6 Final Summary	26
References	27
Annexure I: Viva Preparation Questions	28
Annexure II: Submission Checklist	29

List of Tables

S. No.	Table Name
1	Requirement Summary
2	Functional and Non-Functional Requirements
3	Hardware and Software Requirements
4	Technology Stack
5	Database Collections
6	Roles and Permissions
7	Test Cases Summary
8	Future Enhancements

List of Figures

S. No.	Figure Name
1	High-Level System Architecture
2	Authentication Workflow
3	Suggested Output Screens for Final Submission

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
JWT	JSON Web Token
MERN	MongoDB, Express.js, React.js, Node.js
ODM	Object Data Modeling
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMS	Short Message Service
SPA	Single Page Application
UI/UX	User Interface / User Experience

Chapter 1: Introduction

1.1 Project Overview

The School Management System is a full-stack web application designed to support daily school operations through a single digital platform. Instead of maintaining separate registers, spreadsheets and communication channels, the system keeps core academic and administrative data in one place.

The application is divided into role-based dashboards for administrators, teachers and students. An administrator can manage users, profiles, fees, expenses and academic setup. Teachers can work with attendance, assignments and student academic records. Students can view their own profile, attendance, assignments and results.

1.2 Background of the Problem

A school handles a large amount of information every day: student admissions, teacher records, attendance, fee collections, expenses, assignments, examinations, results and notifications. When these processes are handled manually, the work becomes slow and repetitive. Records may be misplaced, reports take longer to prepare and communication with students or parents is delayed.

A digital management system helps reduce these problems by creating a clear workflow for storing, updating and retrieving information. It also improves transparency because authorized users can access relevant data whenever required.

1.3 Problem Statement

The problem addressed by this project is the absence of a simple, secure and centralized system for handling school administration and academic records. The project aims to develop a web application that reduces manual record-keeping, protects user data, organizes academic workflows and supports communication through notifications.

1.4 Aim of the Project

The aim of the project is to design and develop a School Management System that helps an institution manage students, teachers, attendance, assignments, results, fees and expenses through a modern web interface.

1.5 Objectives of the Project

- Build secure authentication using JWT and Argon2 password hashing.
- Provide separate access for Admin, Teacher and Student users through role-based authorization.
- Create modules for student and teacher profile management.
- Implement attendance, assignment, examination and result management.
- Maintain fee and expense records in a structured way.
- Integrate Cloudinary for file storage and Twilio for SMS-based notifications.

- Generate useful PDF documents and QR codes for verification-related tasks.
- Design a responsive user interface that can work on desktop, tablet and mobile screens.

1.6 Scope of the Project

The scope of the system covers common school administration activities. It includes user management, profile management, academic management, attendance, assignments, results, financial records, notifications and document utilities. The system is designed as a web application with a React frontend and an Express/MongoDB backend.

The project does not attempt to replace every feature of a large commercial ERP. Instead, it focuses on the core modules that are useful for a practical school management prototype.

1.7 Importance and Applicability

The system is useful for institutions that want a clean and affordable way to digitize their records. It is especially suitable for small and medium schools where manual work is still common and where staff need a simple dashboard instead of complex enterprise software.

1.8 Project Type and Category

Field	Details
Project Type	Live / Academic Web Application Project
Project Category	Web-Based Project and Database Project
Development Model	Iterative full-stack development
Primary Stack	MERN stack with Vite-based React frontend

1.9 Organization of the Report

This report is organized into seven chapters. Chapter 1 introduces the project. Chapter 2 discusses existing systems and gaps. Chapter 3 explains requirements. Chapter 4 presents system design. Chapter 5 describes implementation. Chapter 6 covers testing and results. Chapter 7 concludes the report and lists possible future improvements.

Chapter 2: Review of Literature and Existing Systems

2.1 Introduction

This chapter studies the need for school management software and reviews common approaches used by institutions. The focus is not only on available systems, but also on the practical gaps that affect students, teachers and administrative staff.

2.2 Existing Methods

Many schools still use a combination of paper registers, spreadsheet files and manual communication. This works for small amounts of data, but it becomes difficult when the number of students, classes and daily records increases. Updating one record in multiple places also creates a higher chance of errors.

Some institutions use commercial ERP systems. These systems usually include many features, but they can be costly and difficult to customize. Small institutions may not need every feature of a large ERP, and staff may find the interface complicated.

2.3 Related Technologies

Modern web applications are commonly built as single-page applications on the frontend and REST APIs on the backend. React.js helps build reusable interface components, while Node.js and Express.js provide a flexible backend environment. MongoDB is useful where data structures may vary between different types of users and records.

Security is also important in education-related software because student and parent information is sensitive. For this reason, password hashing, token-based authentication, role-based access and validation are key design concerns.

2.4 Comparative Study

The following table summarizes the practical observations from existing approaches and the improvements targeted in this project.

Approach	Strengths	Limitations	How this Project Responds
Paper registers	Easy to start and familiar to staff.	Slow search, risk of loss, manual reporting.	Stores records digitally and allows quicker retrieval.
Spreadsheets	Useful for small datasets and calculations.	Difficult to control access and maintain consistency.	Uses a database with structured models and access rules.
Desktop software	Can work without a browser and may be simple locally.	Limited remote access and harder updates.	Uses a browser-based system with central backend APIs.
Commercial ERP	Feature-rich and scalable.	Often costly, complex and less customizable.	Focuses on practical core modules with a modern stack.

2.5 Research Gap / System Gap

The main gap is the need for a manageable system that covers important school workflows without becoming too complex. A useful solution should include secure authentication, clear user roles, digital profiles, attendance, assignments, examinations, fees and basic communication support.

2.6 Need of the Proposed System

The proposed system is needed because it combines the practical features of a school management platform with the flexibility of a custom MERN application. It can be extended module by module and deployed using common web hosting and database services.

2.7 Summary

The literature and existing system review shows that schools require tools that are simple, secure and easy to adapt. This project addresses those needs by designing a web-based system focused on real administrative and academic tasks.

Chapter 3: Requirement Analysis

3.1 Introduction

Requirement analysis helps define what the system should do and how it should behave. For this project, requirements were collected by studying common school workflows and converting them into software modules.

3.2 Stakeholders

- Administrator: manages the overall system, users, finance and academic setup.
- Teacher: manages attendance, assignments, grading and student academic records.
- Student: views personal academic information and submits assignments.
- Parents: may receive important updates through notifications, depending on the school workflow.

3.3 Requirement Summary

Module	Main Requirement	Expected Outcome
Authentication	Login, registration, JWT tokens and password hashing.	Only verified users can access protected dashboards.
Profile Management	Maintain student, teacher and parent-related details.	Records become easier to update and retrieve.
Attendance	Mark and view attendance records.	Teachers and students can track attendance accurately.
Assignments	Create assignments and accept submissions.	Academic work is stored and reviewed digitally.
Examination	Create exams and record results.	Results become easier to manage and export.
Finance	Track fees and expenses.	Financial records are more transparent.
Notifications	Store alerts and send selected SMS updates.	Important messages reach users faster.
Documents	Generate PDFs and QR-based records.	Reports and ID-related documents can be produced quickly.

3.4 Functional Requirements

- The system shall allow users to log in securely based on their role.
- The administrator shall be able to create and manage student and teacher profiles.
- Teachers shall be able to mark attendance for assigned classes.
- Teachers shall be able to create assignments and review submitted files.
- The system shall store examination and result information.
- The finance module shall record fees and institutional expenses.
- The system shall support file upload for profile images and assignment files.
- The system shall generate PDF reports or documents where required.

- The system shall send selected SMS notifications through an integrated service.

3.5 Non-Functional Requirements

Requirement	Description
Security	Passwords must be hashed, routes must be protected and users must be authorized by role.
Usability	The interface should be simple enough for non-technical school staff.
Performance	Common dashboard and listing pages should load quickly under normal usage.
Maintainability	Backend routes, controllers, models and middleware should be separated clearly.
Scalability	The database structure should allow more classes, students and modules in the future.
Responsiveness	The frontend should work on common desktop and mobile screen sizes.

3.6 Hardware and Software Requirements

Category	Requirement
Processor	Multi-core processor, Intel i5 or equivalent preferred for development.
RAM	8 GB recommended for development; 4 GB may be sufficient for basic deployment.
Storage	20 GB SSD or higher for source code, dependencies and database tools.
Runtime	Node.js version 18 or above.
Database	MongoDB Server or MongoDB Atlas.
Package Manager	npm or equivalent JavaScript package manager.
Browser	Modern browser such as Chrome, Edge or Firefox.
Development Tools	VS Code, Postman or similar API testing tool, Git.

3.7 Use Case Overview

The main use cases are login, manage users, maintain profiles, mark attendance, create assignments, submit assignments, create exams, enter results, track fees, add expenses and send notifications. Each use case is restricted according to the user role so that one user cannot perform another role's administrative tasks.

Chapter 4: System Design

4.1 Introduction

System design explains how the project is structured internally. The application follows a client-server model where the React frontend communicates with the Express backend through REST APIs. MongoDB stores persistent records.

4.2 Proposed System Architecture

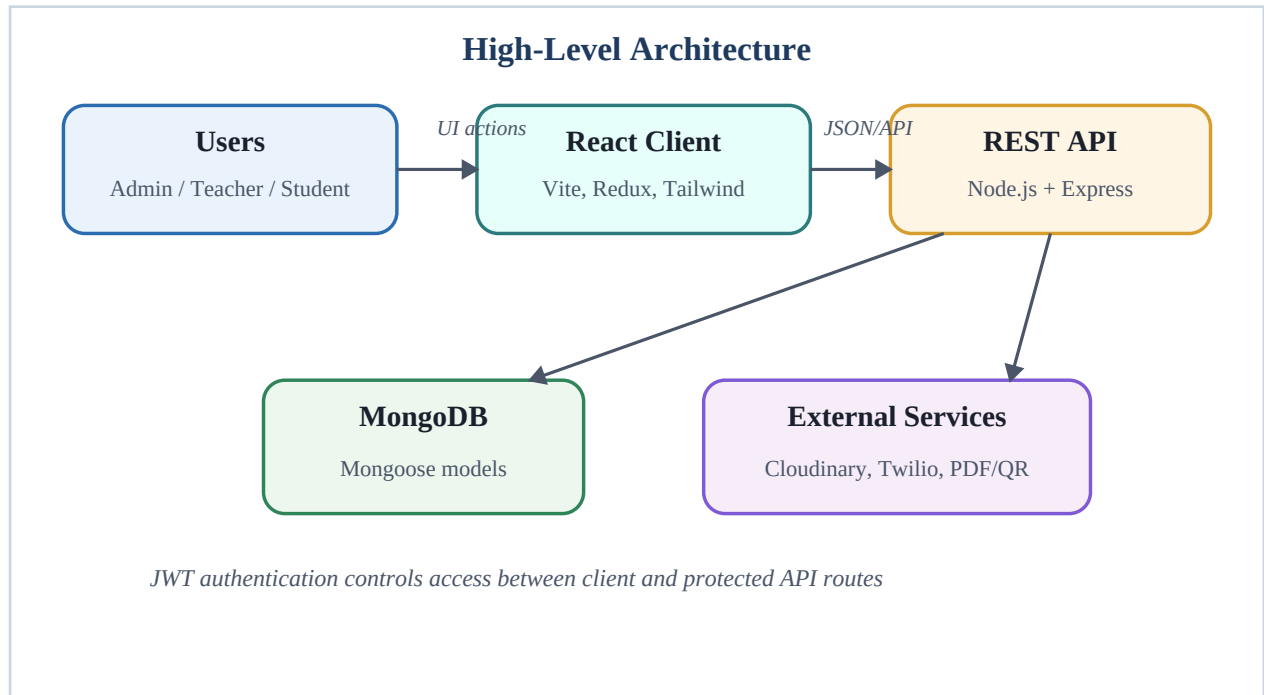


Figure 1: High-Level System Architecture

The frontend handles user interaction, routing, forms and dashboard views. The backend handles authentication, authorization, validation, business logic and database operations. External services are used only where they provide a clear benefit, such as image storage, SMS notifications or document generation.

4.3 Technology Stack

Layer	Technology	Purpose
Frontend	React.js with Vite	Builds a fast single-page interface.
Styling	Tailwind CSS and Shadcn UI	Provides responsive layouts and reusable UI components.
State Management	Redux Toolkit	Manages global user/session and application state.
Form Handling	React Hook Form and Zod	Handles form input and schema-based validation.
Backend	Node.js and Express.js	Creates REST APIs and handles server-side logic.
Database	MongoDB with Mongoose	Stores users, profiles, academic and finance records.
Authentication	JWT and Argon2	Protects accounts and API access.
Media	Multer and Cloudinary	Handles file upload and cloud media storage.
Notifications	Twilio	Sends SMS alerts where needed.
Documents	PDFKit and QRCode	Generates PDFs and QR code based records.

4.4 Database Design

The database uses MongoDB collections with Mongoose schemas. ObjectId references are used where records need to be connected, such as students with attendance, fees, results and submissions.

Collection	Purpose	Important Relationships
Users	Stores login data, role, email and hashed password.	References student, teacher or parent profile.
StudentProfile	Stores admission details, class, parents, contact and academic information.	Linked with attendance, fees, results and submissions.
TeacherProfile	Maintains teacher details, assigned classes and subject responsibilities.	Linked with assignments and attendance marking.
Subjects / Syllabus	Defines curriculum structure and learning units.	Used while creating assignments and exams.
Assignments / Submissions	Tracks work given by teachers and files submitted by students.	Submission documents are stored through Cloudinary.
Attendance	Records daily student presence or absence.	Connected to student profile and class.
Exams / Results	Stores exam setup and marks.	Results may be exported as PDF.
Fees / Expenses	Maintains payment and institutional expense records.	Supports finance dashboard summaries.
Notifications	Stores alerts and system messages.	Can trigger SMS through Twilio.

Table 5: Database Collections

4.5 Role-Based Access Control

Role-based access control is used to make sure that users only access permitted features. This is essential because the system contains sensitive student and financial information.

Role	Allowed Actions	Restricted Actions
Admin	Manage users, profiles, fees, expenses, exams, subjects and system records.	Cannot bypass authentication; all actions still require a valid token.
Teacher	Mark attendance, create assignments, view assigned class records and grade submissions.	Cannot manage all users or view unrelated financial records.
Student	View own profile, attendance, assignments and results; upload assignment submissions.	Cannot edit other students, create exams or access admin modules.

4.6 Data Flow

- 1 The user opens the frontend application and submits login credentials.
- 2 The backend verifies the credentials and generates tokens when the login is valid.
- 3 The frontend stores the session state and sends the token with protected API requests.
- 4 The backend checks the token and user role before running the controller logic.
- 5 The controller reads or updates MongoDB data through Mongoose models.
- 6 The API returns the response to the frontend, where the UI updates accordingly.

Chapter 5: Implementation

5.1 Introduction

The implementation was completed in stages. First, the backend models and APIs were prepared. After that, the frontend pages and components were connected to the APIs. The approach helped test each module gradually instead of building the entire project at once.

5.2 Backend Structure

The backend is organized using a common Express.js structure. Routes define API endpoints, controllers contain business logic, models define MongoDB schemas and middleware handles tasks such as authentication, error handling, validation and file upload.

```
server/  
  controllers/  
  middleware/  
  models/  
  routes/  
  utils/  
  config/  
  app.js  
  server.js
```

5.3 Frontend Structure

The frontend uses React components for pages, forms, dashboards and reusable UI elements. Redux Toolkit is used to manage shared application state such as the logged-in user and fetched records.

```
src/  
  components/  
  pages/  
  redux/  
  services/  
  hooks/  
  utils/  
  App.jsx  
  main.jsx
```

5.4 Authentication Workflow

Authentication is one of the most important parts of the project. The password is never stored as plain text. During registration, it is hashed using Argon2. During login, the entered password is verified against the saved hash. If the credentials are correct, the server generates a JWT for protected routes.

```
// Simplified authentication flow  
const user = await User.findOne({ email });  
if (!user) return res.status(401).json({ message: 'Invalid credentials' });  
  
const isMatch = await argon2.verify(user.password, password);  
if (!isMatch) return res.status(401).json({ message: 'Invalid credentials' });  
  
const token = jwt.sign(  
  { id: user._id, role: user.role },  
  process.env.JWT_SECRET,  
  { expiresIn: '1d' }  
);
```

```
);  
return res.json({ user, token });
```

Figure 2: Authentication Workflow

5.5 Main Modules Implemented

5.5.1 User and Profile Module

This module manages user accounts and profile records. Since different users need different information, separate profile schemas are used for students, teachers and parent-related data.

5.5.2 Attendance Module

The attendance module allows teachers to mark student attendance for a specific class and date. Attendance records are stored in the database and can be viewed later for reporting.

5.5.3 Assignment and Submission Module

Teachers can create assignments and students can upload submissions. Files are first handled by Multer on the backend and then uploaded to Cloudinary. The stored file URL is saved in MongoDB.

5.5.4 Examination and Result Module

The examination module stores exam details and result entries. Results can be displayed in the dashboard and can also be used for generating reports.

5.5.5 Finance Module

The finance module tracks fee collection and institutional expenses. This helps the administration maintain a clear record of payments and outgoing costs.

5.5.6 Notification Module

Notifications are stored in the database and selected alerts may be sent through Twilio SMS. This is useful for important updates such as attendance, fee reminders or academic notices.

5.6 Security Measures

- Passwords are hashed using Argon2 before storage.
- JWT is used to verify protected API requests.
- Role-based middleware prevents users from accessing unauthorized modules.
- Helmet helps secure HTTP headers in the Express application.
- Rate limiting reduces the risk of repeated brute-force requests.
- Form validation helps prevent incomplete or invalid data from entering the system.
- File uploads are handled through a controlled upload process rather than direct unsafe storage.

5.7 Development Methodology

The project followed an iterative development method. Backend APIs were created and tested first. Once the API responses were stable, the frontend components were integrated. This made debugging easier because problems could be checked either at the API level or at the UI level.

Chapter 6: Testing, Results and Discussion

6.1 Introduction

Testing was performed to verify whether the main modules behave as expected. Manual API testing and frontend testing were used to check login, protected routes, form validation, file upload, attendance, assignments and role-based access.

6.2 Output Screens to Attach

The report is prepared with the following output screens in mind. In the final college submission, actual screenshots from the running application should be inserted with these captions.

Figure	Recommended Screenshot	What it Should Show
Figure 3.1	Admin Dashboard	Summary cards, charts and quick access to modules.
Figure 3.2	Student Profile	Student details, class information and parent/contact data.
Figure 3.3	Attendance Page	Class-wise attendance marking and attendance status.
Figure 3.4	Assignment Portal	Assignment creation and submission listing.
Figure 3.5	Fees and Expenses	Fee records, expenses and financial overview.
Figure 3.6	Result / PDF Output	Generated report or result document.

6.3 Test Cases Summary

Test Case	Expected Result	Actual Result	Status
Login with valid user	User receives token and dashboard opens.	Dashboard opened correctly.	Pass
Login with wrong password	Server returns unauthorized response.	401 response received.	Pass
Access admin route as student	Request should be blocked.	403 forbidden response received.	Pass
Submit form with missing fields	Validation error should appear.	Validation message displayed.	Pass
Upload assignment file	File should upload and URL should be saved.	Cloudinary URL saved in database.	Pass
Mark attendance	Attendance record should be stored for selected date.	Record saved and displayed.	Pass
Create fee record	Fee entry should be added to database.	Fee entry visible in finance module.	Pass

6.4 Functional Results

The core modules worked according to the project objectives. Login and role-based access controlled the dashboard views. Profile data, attendance records, assignments, results and finance entries were stored in MongoDB. File upload through Cloudinary and PDF generation through PDFKit added practical value to the system.

6.5 User Interface Results

The frontend interface was designed using Tailwind CSS and Shadcn UI components. This helped keep the design clean and consistent across forms, tables and dashboard pages. The use of reusable components also made the frontend easier to maintain.

6.6 Performance Observations

During normal development testing, the Vite frontend provided fast reloads and a smooth development experience. Backend routes responded properly for common CRUD operations. MongoDB queries were suitable for the project scale, while aggregation can be used further for dashboard analytics.

6.7 Discussion

The project demonstrates that a school management system can be built with a modern JavaScript stack while keeping the structure understandable. The separation between frontend, backend and database improves maintainability. The use of external services, such as Twilio and Cloudinary, reduces the need to build everything from scratch, although it also introduces cost and dependency considerations.

6.8 Limitations

- The current version depends on external services such as Cloudinary and Twilio.
- Advanced analytics and reporting are limited and can be improved.
- A full production rollout would require stronger monitoring and security testing.
- Offline support is not available in the current version.
- Actual deployment configuration may differ based on hosting provider and database setup.

Chapter 7: Conclusion and Future Scope

7.1 Conclusion

The School Management System was developed as a practical full-stack web application for managing important school operations. It provides separate role-based access for administrators, teachers and students, and covers modules such as profiles, attendance, assignments, results, fees, expenses, notifications and document generation.

The project helped apply the MERN stack in a real-world style application. React and Vite were used for the frontend, while Node.js, Express.js and MongoDB handled the backend and database. Security features such as Argon2 password hashing, JWT authentication and role-based checks were included to protect user data.

The final system is suitable as an academic project and as a working prototype. With additional testing, deployment hardening and feature expansion, it can be developed further into a more complete school administration platform.

7.2 Achievement of Objectives

- A role-based authentication system was implemented.
- Student, teacher and parent-related profile management was structured.
- Attendance, assignments, examinations and results were included as academic modules.
- Fees and expenses were managed through a finance module.
- Cloudinary, Twilio, PDFKit and QRCode were integrated for practical utility features.
- A responsive frontend was designed using modern React tooling.

7.3 Major Learnings

The project improved understanding of full-stack development, especially API design, frontend-backend integration, MongoDB schema relationships, file upload handling, authentication and dashboard development. It also showed the importance of writing clean code structure and testing features module by module.

7.4 Practical Applicability

The system can be used as a base for a school that wants to digitize important administrative and academic tasks. It can also be modified for coaching institutes, training centers or small educational organizations with similar workflows.

7.5 Future Scope

Enhancement	Description
Online Fee Payment	Integrate Razorpay, Stripe or another payment gateway for direct parent payments.
Progressive Web App	Add PWA support so the application feels more like a mobile app and supports limited offline use.
Advanced Analytics	Use dashboard analytics to understand attendance trends, fee status and academic performance.
Parent Portal	Create a separate parent dashboard for attendance, results, fee status and notices.
Email Notifications	Add email alerts along with SMS notifications.
Audit Logs	Maintain logs for sensitive actions such as fee updates and user changes.
Deployment Monitoring	Add logging, error tracking and uptime monitoring for production use.

7.6 Final Summary

The School Management System provides a focused solution for digitizing core school workflows. It is not only a software project, but also a practical exercise in planning, designing, developing and documenting a complete web application.

References

- 1 Node.js Official Documentation, <https://nodejs.org/>, Accessed: _____.
- 2 React Official Documentation, <https://react.dev/>, Accessed: _____.
- 3 Express.js Official Documentation, <https://expressjs.com/>, Accessed: _____.
- 4 MongoDB Documentation, <https://www.mongodb.com/docs/>, Accessed: _____.
- 5 Mongoose ODM Documentation, <https://mongoosejs.com/>, Accessed: _____.
- 6 Redux Toolkit Documentation, <https://redux-toolkit.js.org/>, Accessed: _____.
- 7 Tailwind CSS Documentation, <https://tailwindcss.com/>, Accessed: _____.
- 8 Shadcn UI Documentation, <https://ui.shadcn.com/>, Accessed: _____.
- 9 Argon2 npm package documentation, <https://www.npmjs.com/package/argon2>, Accessed: _____.
- 10 Cloudinary Documentation, <https://cloudinary.com/documentation>, Accessed: _____.
- 11 Twilio Documentation, <https://www.twilio.com/docs>, Accessed: _____.
- 12 PDFKit Documentation, <https://pdfkit.org/>, Accessed: _____.

Annexure I: Viva Preparation Questions

Q: What is the main purpose of your project?

A: The main purpose is to digitize school administration and academic records through a secure web-based system.

Q: Why did you use the MERN stack?

A: The MERN stack allows full-stack JavaScript development. It is suitable for building dynamic interfaces, REST APIs and flexible database models.

Q: How are passwords secured?

A: Passwords are hashed using Argon2 before being stored in the database. During login, the entered password is verified against the stored hash.

Q: What is the role of JWT in your project?

A: JWT is used to verify authenticated users and protect API routes without storing session data on the server.

Q: How is role-based access handled?

A: The user role is stored with the user account and checked in backend middleware before allowing access to protected features.

Q: Why did you use MongoDB?

A: MongoDB is flexible for handling different types of records such as profiles, attendance, assignments, submissions and finance data.

Q: How do file uploads work?

A: Multer receives uploaded files on the backend. The files are then uploaded to Cloudinary and the resulting URL is saved in MongoDB.

Q: What are the main modules?

A: The main modules are authentication, profiles, attendance, assignments, exams, results, fees, expenses, notifications and document generation.

Q: What are the limitations of the system?

A: The system depends on external services and requires further testing, monitoring and deployment hardening for production use.

Q: What future features can be added?

A: Online fee payments, parent portal, PWA support, analytics, email notifications and audit logs can be added.

Annexure II: Submission Checklist

Before submitting the final report, the following items should be checked and completed.

- Fill the student name, registration number, programme, guide name, date and institution details before final submission.
- Add actual screenshots of the running project in the output section.
- Confirm all blank personal and academic details before printing or uploading.
- Update references if additional libraries or tools were used.
- Attach source code or deployment link if required by the department.
- Take a final print preview to ensure all tables and headings are properly aligned.